CANDY
DIALOGUE ENGINE

Translations Guide

1.0.0

Translations are done two different ways depending if you are translating a Spoken Line or parts of the UI.

Default Language

One important thing to keep in mind is that your game's original language is called "Default". Whether the original language is English, French, Italian, Japanese or anything else, Candy DE refers to it as "Default".

Other languages are referred to by the names you give them.

The language can be changed inside **Candy_Properties.gd** at runtime, in the **language** variable.

Spoken Lines

Spoken Lines can carry their own translations for multiple languages, which can be written to the line through the companion Candy Dialogue Creator.

See the **Variants and Translations** guide for more information.

UI

Texts that are part of UI elements are translated through dictionaries in Candy DE. Specifically, these dictionaries are found in **Candy_Localizations.gd**.

One dictionary is provided for each text element, and they all follow the same format:

```
var dictionary = {
      "Text or Reference": {
            "Default": "Text for Default language.",  (Optional)
            "Language 2": "Text for Language 2",
            "Language 3": "Text for Language 3",
            (…)
      }
}
```

The code that makes use of these dictionaries is located directly inside the corresponding UI elements. By default, four types of UI elements use these dictionaries: Input UIs, Choice Menus, Choice Categories, and Choice Buttons.

Unless you are creating your own custom UI elements, you don't need to worry about the code: you'll only deal with localization dictionaries through the §Input command or the §Choice_List command.

As an example, the §Choice_List command allows you to provide a title and a prompt for a choice menu, and labels for choices.

You can provide the exact title/prompt/labels you want to display in your game's Default language. Then, in the corresponding localization dictionaries, add a key for each title/prompt/label, and add sub-keys for each language. Assign each language sub-key the text it should display.

Adding a sub-key for "Default" is optional: if the "Default" sub-key doesn't exist, the code will use the original text provided to the command. You would only need to add a "Default" sub-key if you want to display a different text than the original.

For example, if you provide "Merchant Services" as a title, and the "Merchant Services" key in the **choice_menu_titles** dictionaries doesn't have a "Default" sub-key, the code will display "Merchant Services" on screen if the language is set to Default.

Note that this applies to all languages: for example, if the "Italian" sub-key is missing, and **language** is set to Italian, "Merchant Services" would also be displayed.

Actor Data

Dictionaries are also provided for actor data in **Candy_Localizations.gd**, such as **display_names**. These dictionaries work similarly to those for UI translations.

The §Name command (used for changing an actor's display name) will automatically call the **translate()** function in **Candy_Engine.gd**.

translate() can be used to look up any value in any translation dictionary, and return the correct translation (if it exists; otherwise, it returns the original value).

When called by the §Name command, this function will lookup the provided display name in the **display_names** dictionary, and check if a translation exists for the current language. If so, it will assign that translation to the actor as its display name; if not, it will assign the original display name provided to the §Name command.

Other Uses

You can add your own dictionaries to the **Candy_Localizations.gd** script for use in your custom UIs or for other custom purposes. Note that you'll need to write code to use these dictionaries, but you can probably use the code provided in the default UI elements as a template, or you can call the **translate()** function for help.

translate() has the advantage of being able to interpret variable references (e.g. £candy_de.display_names) as the translation dictionary it should use; on top of that, if it's provided

a variable without a singleton or node prefix (e.g. 'display_names' instead of £candy_de .display_names), it assumes that it's a dictionary inside the 'candy_de' singleton.

Language Change

If players have the ability to change the game language while Candy DE UI elements with text are displayed, you should write code to update the text in those UI elements. However, this isn't necessary if players can't change the language while these UI elements are visible.

Remember to also write code to translate actor data when language changes, such as display names.

Do not translate the "Gender" key, as it's meant for internal (technical) use by the code, not for display: using different values across languages could break some Candy DE functionalities. If you want to display a character's gender in text, you should create and use a different key (e.g. "Display Gender").

External Translation Work

You don't have to write translations directly in the **Candy_Localizations.gd** script. If you don't want your translators to access the script directly, you can let them modify copies of the dictionaries in separate files, then copy/paste to **Candy_Localizations.gd**.

Alternately, you can also have translators write to a .csv or other file format, and use a custom function in your game that will read from such files and update the localization dictionaries.

In the future, and with the guidance of user feedback, we may add tools to the Candy Dialogue Creator to create such translation files, in conjunction with Candy DE code to update the localization dictionaries from these files.

Changing the translation methods described in this guide may be difficult (we'd rather avoid heavy engine redesigns and changes that could break things for users), but we're open to suggestions and feedback: we know how daunting translating a game can be, and we want to make the process as easy and efficient as possible.